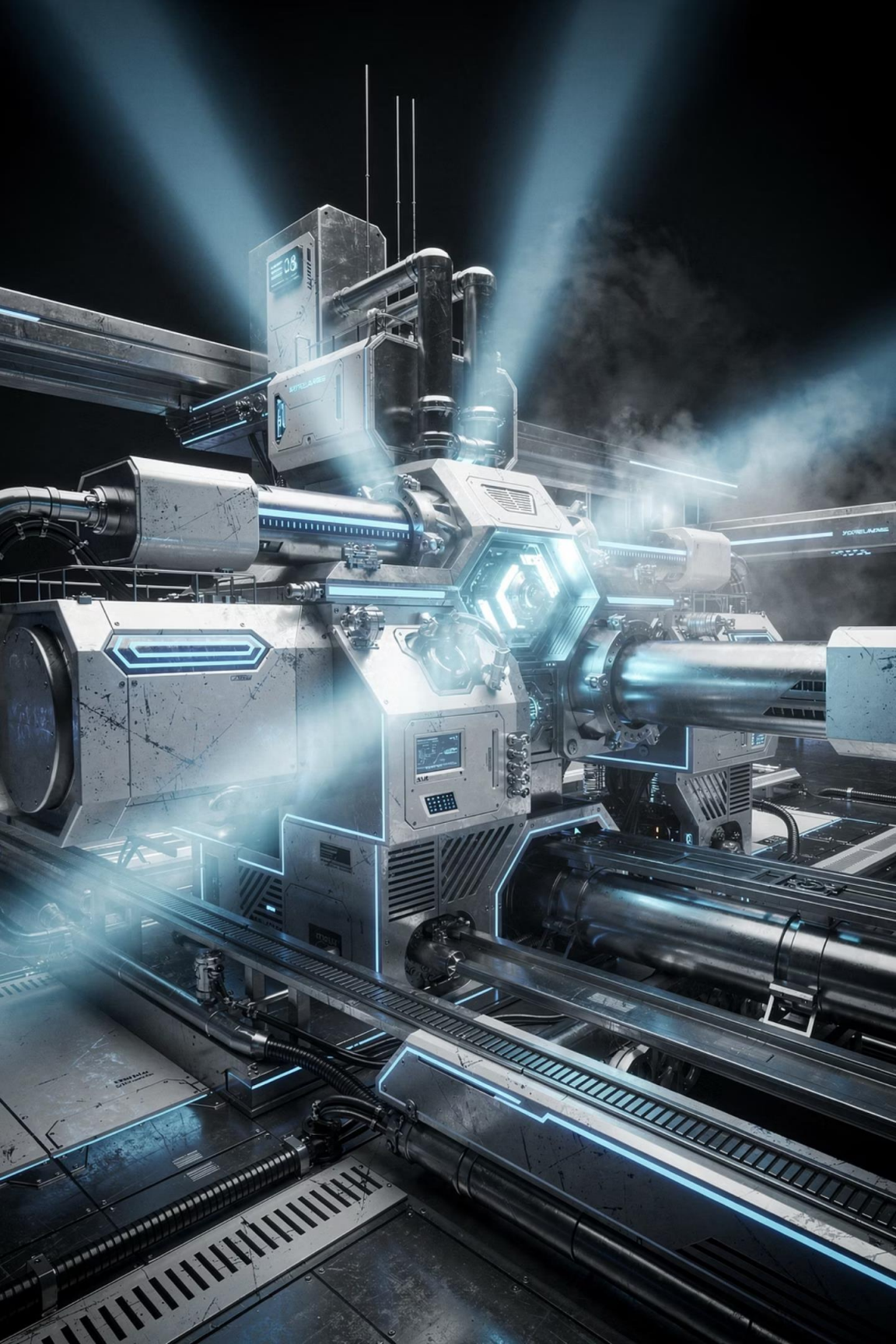




# Factory Method

Método fábrica · Constructor virtual

Un patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclasses alterar el tipo de objetos que se crearán.

# Agenda

01

---

## El Problema

Por qué la creación directa de objetos genera acoplamiento

03

---

## Estructura

Componentes del patrón y sus roles

05

---

## Aplicabilidad

Cuándo y cómo implementarlo

02

---

## La Solución

Cómo el Factory Method resuelve el problema

04

---

## Pseudocódigo

Ejemplo práctico multiplataforma

06

---

## Pros, Contras y Relaciones

Ventajas, desventajas y patrones relacionados

CAPÍTULO 1

# El Problema

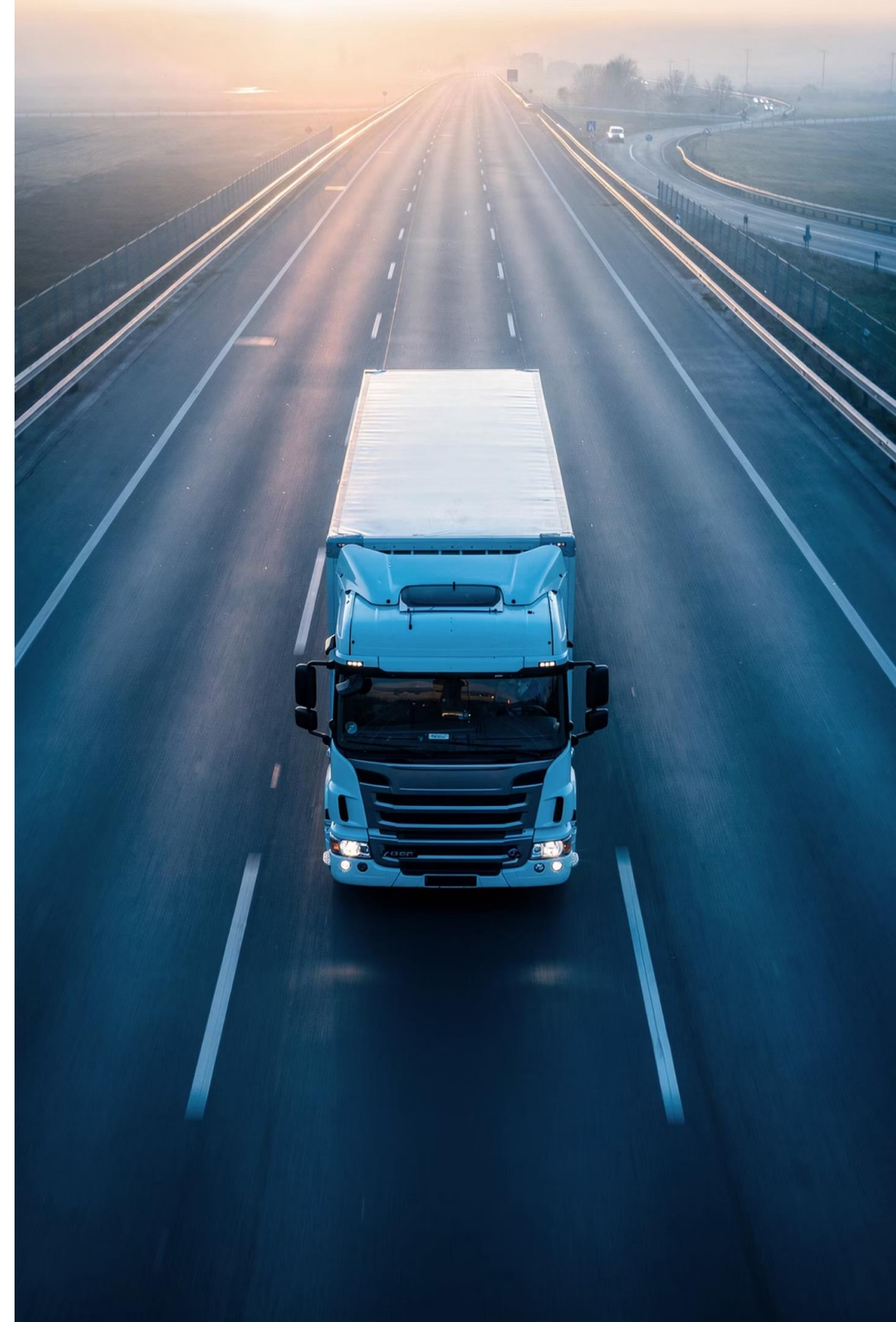
Cuando el código se acopla a clases concretas



# Imagina una aplicación de logística

La primera versión de tu aplicación sólo maneja el transporte en camión, por lo que la mayor parte del código se encuentra dentro de la clase `Camión`.

Al cabo de un tiempo, la aplicación se vuelve popular. Cada día recibes decenas de peticiones de empresas de transporte marítimo para incorporar la logística por mar.

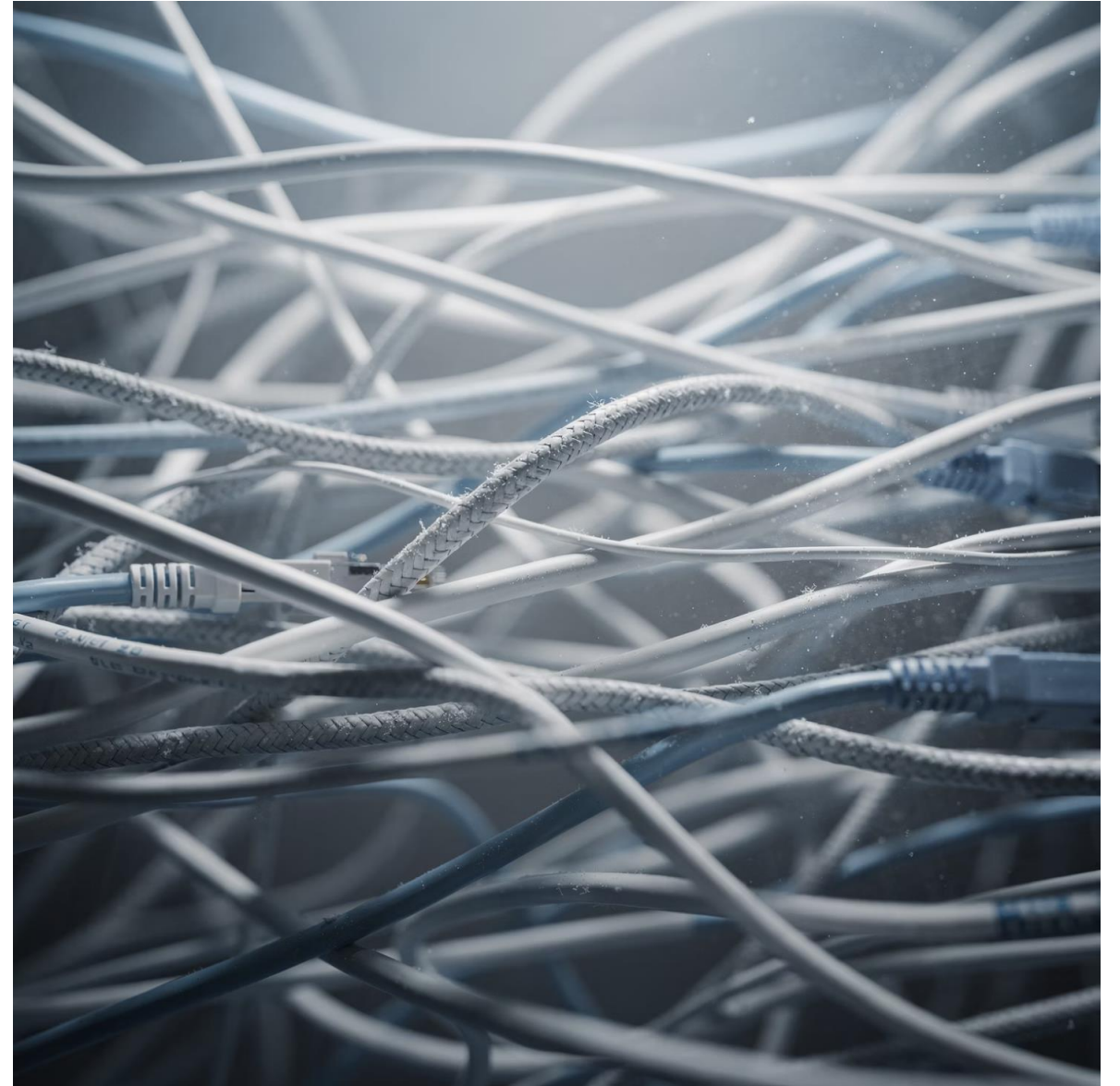


# El código acoplado es un problema

Añadir una nueva clase al programa no es tan sencillo si el resto del código ya está acoplado a clases existentes.

- La mayor parte del código está acoplado a la clase `Camión`
- Para añadir barcos habría que hacer cambios en **toda la base del código**
- Si decides añadir otro tipo de transporte, tendrás que repetir todos estos cambios

❏ Al final acabarás con un código plagado de condicionales que cambian el comportamiento dependiendo de la clase de los objetos de transporte.



CAPÍTULO 2

# La Solución

El patrón Factory Method al rescate



# El método fábrica especial

El patrón Factory Method sugiere que, en lugar de llamar al operador `new` para construir objetos directamente, se invoque a un **método fábrica especial**. Los objetos se siguen creando con `new`, pero se invocan desde el método fábrica.

Los objetos devueltos por el método fábrica a menudo se denominan **productos**.



# Las subclases alteran los productos

Ahora puedes sobrescribir el método fábrica en una subclase y cambiar la clase de los productos creados. Sin embargo, hay una limitación importante:

## Interfaz común

Las subclases sólo pueden devolver productos de distintos tipos si tienen una clase base o interfaz común.

## Tipo de retorno

El método fábrica en la clase base debe tener su tipo de retorno declarado como dicha interfaz.



# Ejemplo: Logística de transporte

## La clave

Tanto `Camión` como `Barco` implementan la interfaz `Transporte`, que declara el método `entrega`.

Cada clase implementa este método de forma diferente.

## LogísticaTerrestre

Su método fábrica devuelve objetos de tipo `Camión`. Entrega por tierra.

## LogísticaMarítima

Su método fábrica devuelve objetos de tipo `Barco`. Entrega por mar.

# El código cliente no nota la diferencia

El código que utiliza el método fábrica (código cliente) no encuentra diferencias entre los productos devueltos por varias subclases. Trata a todos los productos como la clase abstracta `Transporte`.

El cliente sabe que todos los objetos de transporte deben tener el método `entrega`, pero no necesita saber cómo funciona exactamente.

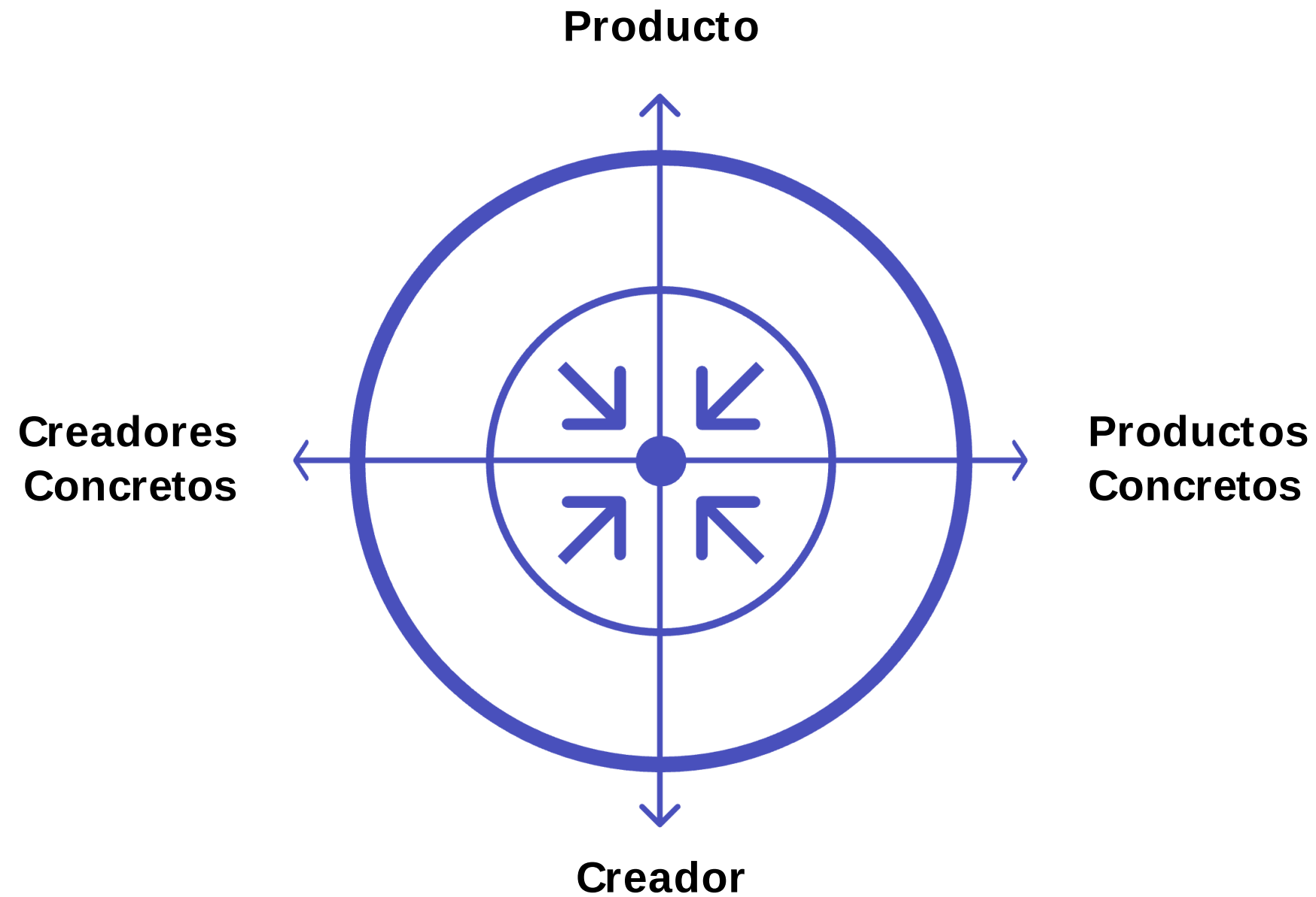


CAPÍTULO 3

# Estructura

Los componentes del patrón Factory Method

# Componentes del patrón



Estos cuatro elementos trabajan juntos para desacoplar la creación de objetos de su uso concreto.



# Detalle de cada componente

1

## Producto

Declara la interfaz común a todos los objetos que puede producir la clase creadora y sus subclases.

2

## Productos Concretos

Son distintas implementaciones de la interfaz de producto.

3

## Clase Creadora

Declara el método fábrica que devuelve nuevos objetos de producto. El tipo de retorno debe coincidir con la interfaz de producto.

4

## Creadores Concretos

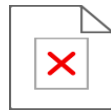
Sobrescriben el Factory Method base para devolver un tipo diferente de producto.

# Matrices importantes de la Creadora



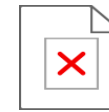
## Puede ser abstracto

Puedes declarar el Factory Method como abstracto para forzar a todas las subclases a implementar sus propias versiones.



## Responsabilidad principal

La creación de producto **no** es la principal responsabilidad de la creadora. Normalmente contiene lógica de negocios central relacionada con los productos.



## Reutilización

El método fábrica no tiene que crear nuevas instancias siempre. Puede devolver objetos existentes de una caché u otra fuente.

CAPÍTULO 4

# Pseudocódigo

Ejemplo práctico: UI multiplataforma

# Diálogo multiplataforma

Este ejemplo ilustra cómo utilizar el Factory Method para crear elementos UI multiplataforma sin acoplar el código cliente a clases UI concretas.

La clase base de diálogo utiliza distintos elementos UI para representar su ventana. En varios sistemas operativos, estos elementos pueden tener aspectos diferentes, pero su **comportamiento debe ser consistente**. Un botón en Windows sigue siendo un botón en Linux.



# ¿Cómo funciona?

1

## Clase base Dialog

Declara un Factory Method  
`createButton()` que produce botones.

2

## Subclase WindowsDialog

Devuelve botones al estilo Windows  
desde el Factory Method.

3

## Subclase WebDialog

Devuelve botones HTML desde el Factory  
Method.

La subclase hereda la mayor parte del código del diálogo de la clase base, pero gracias al Factory Method, puede representar botones específicos de cada plataforma.



# Interfaz de botones abstractos

## Requisito clave

La clase base de diálogo debe funcionar con **botones abstractos**: una clase base o interfaz que sigan todos los botones concretos.

De este modo, el código sigue siendo funcional independientemente del tipo de botones con el que trabaje.

- ❏ Con cada nuevo método de fábrica que añadas al diálogo, más te acercará al patrón **Abstract Factory**.



# Pseudocódigo: La clase creadora

```
class Dialog is
  abstract method createButton():Button

  method render() is
    // Invoca el método fábrica
    Button okButton = createButton()
    // Ahora utiliza el producto
    okButton.onClick(closeDialog)
    okButton.render()
```

La creadora declara el método fábrica abstracto y contiene la lógica de negocio que depende de los productos devueltos.

# Pseudocódigo: Creadores concretos

## WindowsDialog

```
class WindowsDialog extends Dialog is  
  method createButton():Button is  
    return new WindowsButton()
```

## WebDialog

```
class WebDialog extends Dialog is  
  method createButton():Button is  
    return new HTMLButton()
```

Cada creador concreto sobrescribe el método fábrica para devolver su tipo de producto específico.

# Pseudocódigo: Productos

```
interface Button is  
    method render()  
    method onClick(f)
```

```
class WindowsButton implements Button is  
    method render(a, b) is  
        // Representa un botón en estilo Windows.  
    method onClick(f) is  
        // Vincula un evento clic de OS nativo.
```

```
class HTMLButton implements Button is  
    method render(a, b) is  
        // Devuelve una representación HTML de un botón.  
    method onClick(f) is  
        // Vincula un evento clic de navegador web.
```

# Pseudocódigo: La aplicación

```
class Application is
  field dialog: Dialog

  method initialize() is
    config = readApplicationConfigFile()
    if (config.OS == "Windows") then
      dialog = new WindowsDialog()
    else if (config.OS == "Web") then
      dialog = new WebDialog()
    else
      throw new Exception("Error! Unknown OS.")

  method main() is
    this.initialize()
    dialog.render()
```

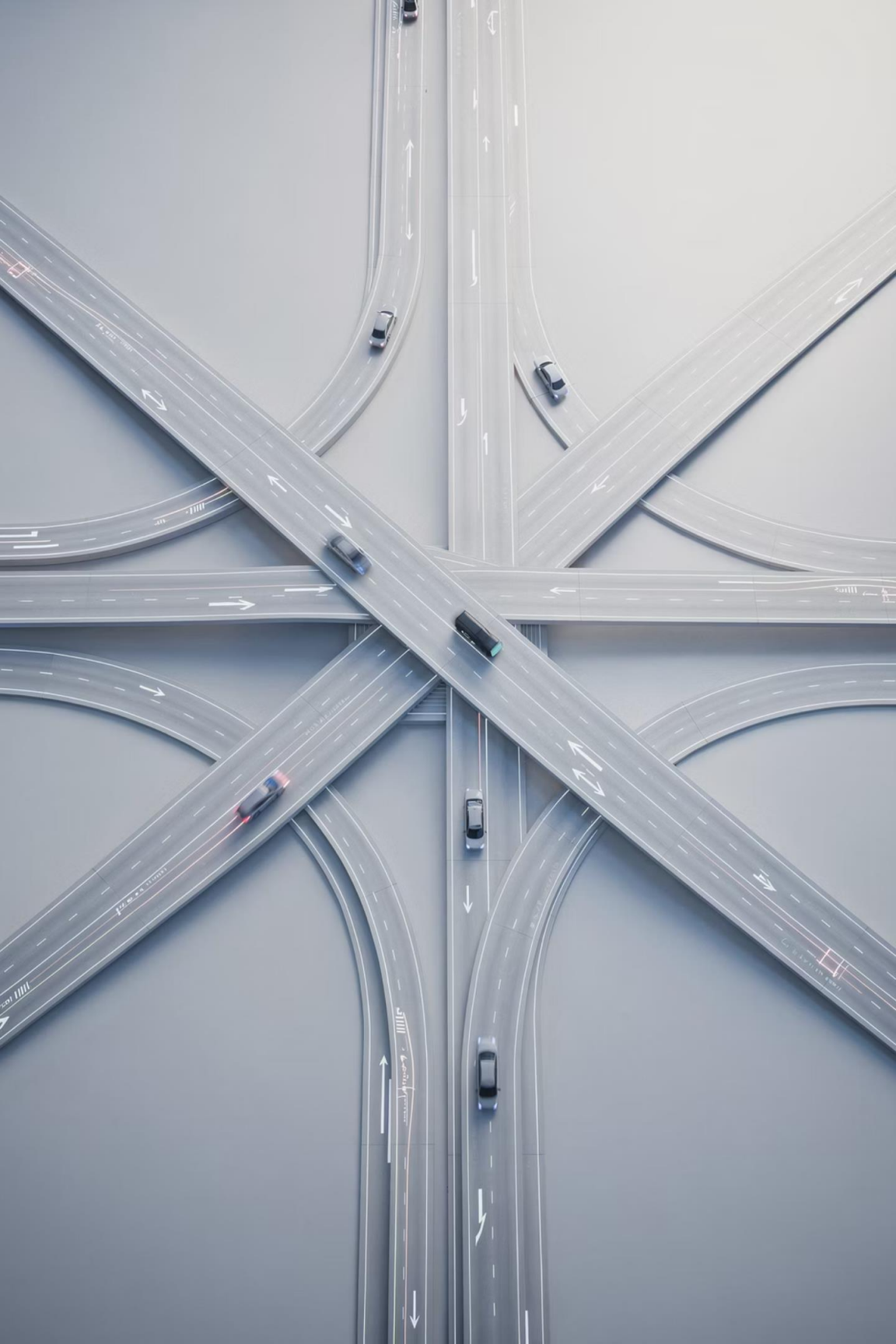
La aplicación elige un tipo de creador dependiendo de la configuración. El código cliente funciona con el creador a través de su interfaz base.



CAPÍTULO 5

# Aplicabilidad

¿Cuándo usar el Factory Method?



# Caso 1: Dependencias desconocidas

Utiliza el Factory Method cuando **no conozcas de antemano** las dependencias y los tipos exactos de los objetos con los que deba funcionar tu código.

El patrón separa el código de construcción de producto del código que hace uso del producto. Es más fácil extender el código de construcción de forma independiente al resto. Para añadir un nuevo tipo de producto, sólo creas una nueva subclase creadora y sobrescribes el Factory Method.

# Caso 2: Extensibilidad de bibliotecas

Utiliza el Factory Method cuando quieras ofrecer a los usuarios de tu biblioteca o framework una forma de **extender sus componentes internos**.

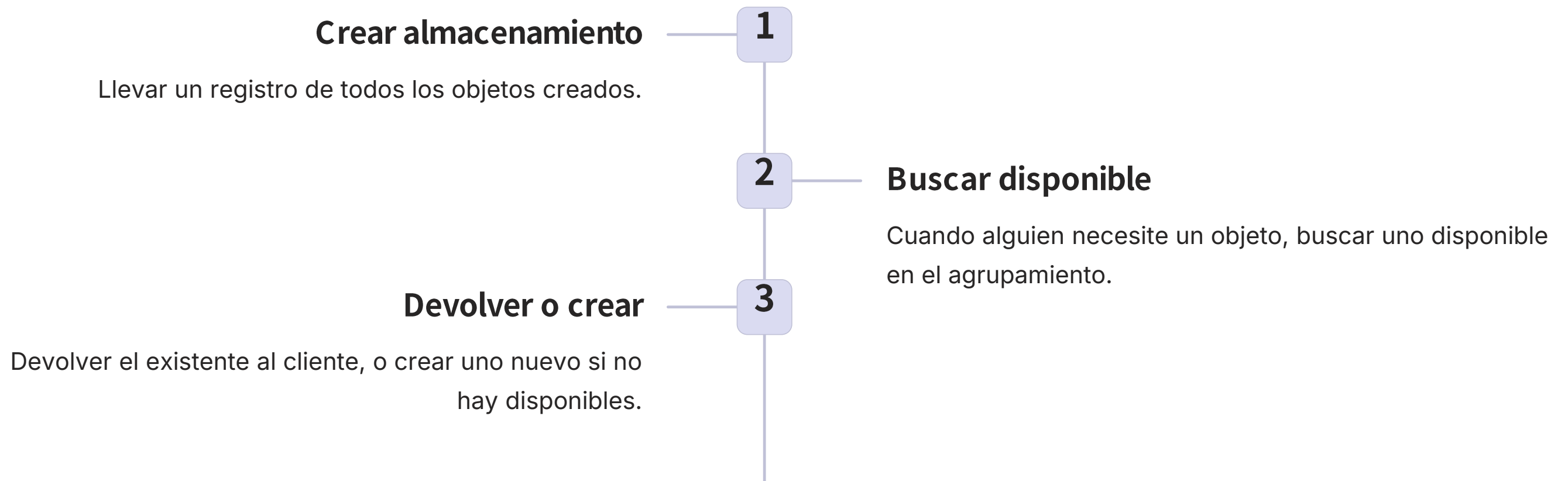
**Ejemplo:** Tu aplicación necesita botones redondos, pero el framework sólo proporciona cuadrados.

- 1** Extiende `Botón` con `BotónRedondo`
- 2** Crea subclase `UIConBotonesRedondos`
- 3** Sobrescribe `crearBotón` para devolver `BotónRedondo`



# Caso 3: Reutilización de recursos

Utiliza el Factory Method cuando quieras **ahorrar recursos del sistema** mediante la reutilización de objetos existentes en lugar de reconstruirlos cada vez.



Un constructor siempre debe devolver nuevos objetos por definición. Necesitas un método regular capaz de crear nuevos objetos y reutilizar los existentes: eso es un Factory Method.

CAPÍTULO 6

# Implementación

Paso a paso para aplicar el patrón

# Cómo implementarlo

## Paso 1: Interfaz común

Haz que todos los productos sigan la misma interfaz con métodos que tengan sentido en todos los productos.

## Paso 2: Factory Method vacío

Añade un Factory Method vacío dentro de la clase creadora. El tipo de retorno debe coincidir con la interfaz común.

## Paso 3: Sustituir constructores

Encuentra todas las referencias a constructores de producto y sustitúyelas por invocaciones al Factory Method.



# Pasos finales de implementación

1

## Crear subclases creadoras

Crea un grupo de subclases creadoras para cada tipo de producto. Sobrescribe el Factory Method en las subclases y extrae las partes adecuadas de código constructor.

2

## Reutilizar parámetros

Si hay demasiados tipos de producto, puedes reutilizar el parámetro de control de la clase base. Ejemplo: `CorreoTerrestre` puede recibir un argumento para controlar si devuelve `Camión` o `Tren`.

3

## Refinar el método base

Si tras las extracciones el Factory Method base queda vacío, hazlo abstracto. Si queda algo, conviértelo en comportamiento por defecto.

CAPÍTULO 7

# Pros y Contras

Evaluando el patrón Factory Method



# Ventajas y desventajas

## ✓ Ventajas

- **Desacoplamiento:** Evitas un acoplamiento fuerte entre el creador y los productos concretos.
- **Responsabilidad única:** Puedes mover el código de creación a un lugar del programa, facilitando el mantenimiento.
- **Abierto/cerrado:** Puedes incorporar nuevos tipos de productos sin descomponer el código cliente existente.

## ✗ Desventajas

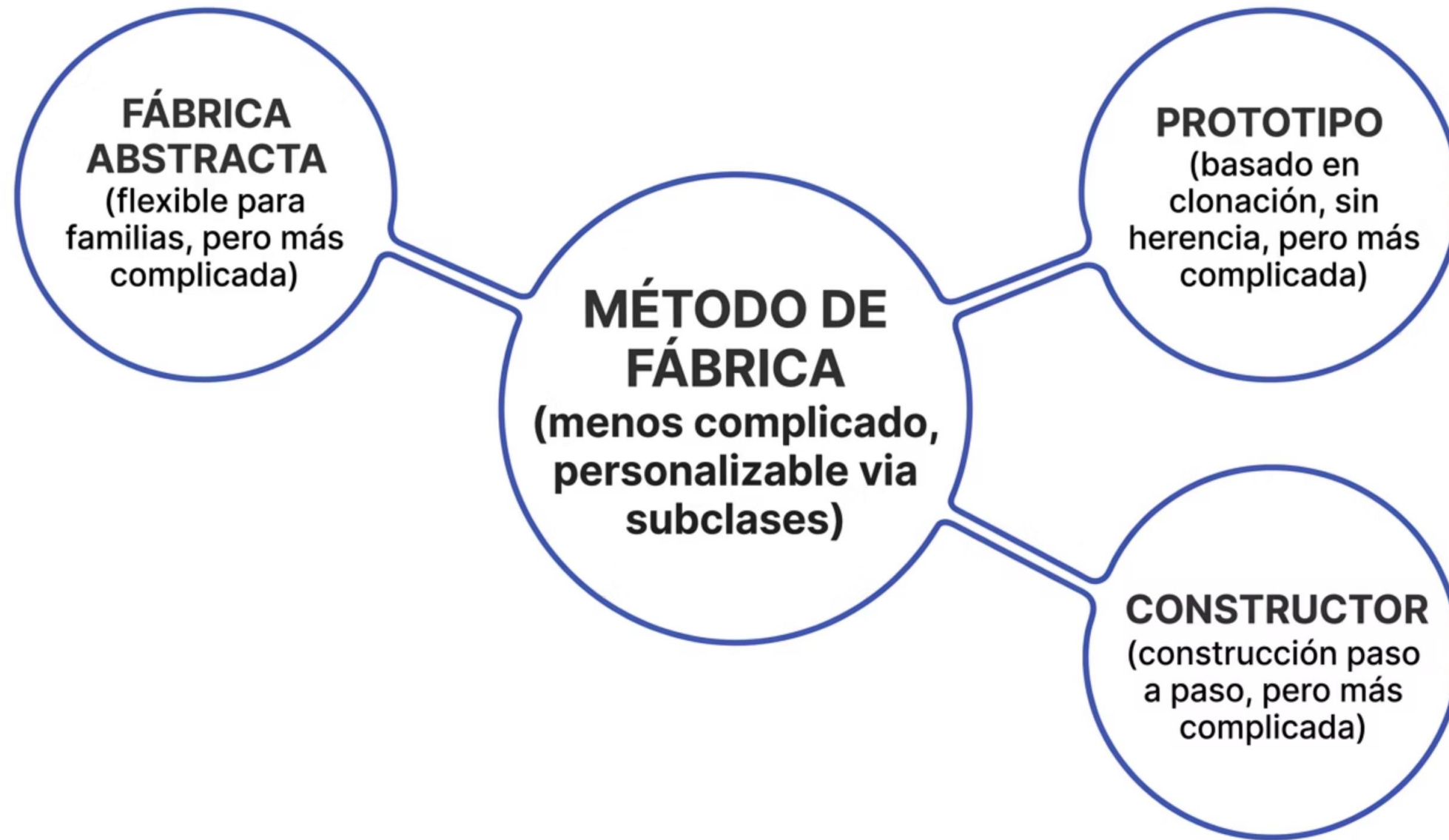
- **Complejidad:** El código puede complicarse al incorporar una multitud de nuevas subclases para implementar el patrón.
- La situación ideal sería introducir el patrón en una **jerarquía existente** de clases creadoras.

CAPÍTULO 8

# Relaciones

Conexiones con otros patrones de diseño

# Evolución desde Factory Method



Muchos diseños empiezan con Factory Method (menos complicado y más personalizable mediante subclases) y evolucionan hacia Abstract Factory, Prototype o Builder.

# Relaciones clave con otros patrones

## Abstract Factory

Sus clases a menudo se basan en un grupo de métodos de fábrica, pero también puedes utilizar Prototype para escribir los métodos de estas clases.

## Iterator

Puedes usar Factory Method junto con Iterator para que las subclases de la colección devuelvan distintos tipos de iteradores compatibles.

## Prototype

No se basa en la herencia (sin sus inconvenientes), pero requiere inicialización complicada del objeto clonado. Factory Method se basa en herencia pero no requiere ese paso.

## Template Method

Factory Method es una especialización del Template Method. Al mismo tiempo, un Factory Method puede servir como paso de un gran Template Method.





# Resumen: Factory Method



## Interfaz común

Todos los productos comparten una interfaz



## Subclases deciden

Cada subclase elige qué producto crear



## Desacoplamiento

Código cliente independiente de clases concretas



## Extensible

Nuevos productos sin romper código existente

El Factory Method es el punto de partida ideal para desacoplar la creación de objetos. Desde aquí, puedes evolucionar hacia patrones más complejos según las necesidades de tu sistema.